uid100999-provenance-investigation

T028 — Forensic investigation: 51 items at uid 100999 vs 0 discovered

Revision: 1 **Last modified:** 2026-08-26T11:20:00Z **Scope:** read-only forensic investigation of the contradiction five T028 review rounds recorded as UNKNOWN. No source was changed; `ownership_repair.sh` was never invoked; nothing was chowned. **Label:** (T11/002-user-owned-downloads - milos85vasic - ? - xhigh)

---

1. The provenance record, verbatim

`config/owned_paths.yaml`, PROVENANCE block (current revision):

```
# ── PROVENANCE (measured 2026-08-21, research.md R6) ──────────────────────
#
#   download root  : owned by uid 100999, renders as UNKNOWN:UNKNOWN
#                    6458 items at uid 1000, 1 at 100999
#   config/        : 51 items at uid 100999
#   config/boba.db : mode 600, owner does not resolve -> ...
```

Its cited source, `specs/002-user-owned-downloads/research.md` §R6, verbatim:

```
## R6. Scope is wider than downloads

**Evidence**:

config/            : 51 items owned by uid 100999
```

```
config/boba.db     : UNKNOWN:UNKNOWN, mode 600
tmp/               : uid 1000 (already correct)
download root      : uid 100999 (the root itself)
downloads content  : 6458 items at uid 1000, 1 at 100999
```

Stated scope: the literal path `config/`, repo-relative. **Stated date:** 2026-08-21. Landed in `6fe921b` (2026-08-21 15:49:31 +0200) and `66effba` (2026-08-21 16:06:12 +0200).

FINDING — the 51 has no recorded derivation. R1 and R2 of the same document paste the exact commands and their outputs (`podman run …`, `stat -c '%u (%U)'`). R6 pastes a results table and **no command**. Machine-checked: an `awk` extraction of the R6 block scanned for `^$`, `find`, `stat`, `awk`, `sort` and returned `NO COMMAND RECORDED IN R6`. The figure therefore cannot be re-derived, re-scoped, or audited from the record.

---

2. Present-day measurement (2026-08-26)

Instrument: `find <path> -printf '%U\n'`, stderr redirected to a file and printed.

Location	entries walked	uid 1000	uid 100999	stderr
`config/`	264	264	**0**	empty
`/run/media/milosvasic/DATA4TB/Downloads` (declared download root)	7681	7681	**0**	empty

`config` itself: `uid=1000 (milosvasic) gid=1000 mode=755`. Download root itself: `uid=1000 (milosvasic) gid=1000 mode=777`.

2.1 Control needle (§11.4.201(7)(b))

A zero from a blind instrument and a zero from a clean tree are identical, so the instrument was proven able to see **the exact target uid** before the zero was reported. Same binary, same predicate, same `-printf '%U\n'`:

```
$ find "$(podman info --format '{{.Store.GraphRoot}}')" -printf
'%U\n' | sort | uniq -c | sort -rn
1239359 1000
    486 100998
    408 100069
    135 100005
     35 100999      <-- the target uid, seen
     26 100502
     ...
```

The instrument resolves 16 distinct uids including **35 items at uid 100999**. It is not blind, and specifically not blind to 100999.

Honest limit of the needle (§11.4.6). The needle ran on `/home` (btrfs, subvol `@home`); the targets are on `/run/media/milosvasic/DATA4TB` (btrfs, `/dev/nvme0n1p1`) — different mounts. An unprivileged user cannot create a uid-100999 file on the target mount, so no needle was placed there. The filesystem-level false-null is nevertheless refuted independently: `findmnt` confirms both targets are **btrfs**, a POSIX filesystem that stores and returns real owner uids (unlike exFAT/NTFS/vfat, which would report the mounting user unconditionally and make every reading structurally meaningless).

---

3. The journal, verbatim — both real runs

Run 1 — 2026-08-21T17:01:55+02:00 (`journalctl --user -u boba-stack.service`):

```
[ownership-repair] operator 1000:1000; scope /run/media/
milosvasic/DATA4TB/Projects/boba/config/owned_paths.yaml (3
declared locations)
[ownership-repair] 1/3 config/boba.db: 0/0 items need repair
[ownership-repair] 2/3 /run/media/milosvasic/DATA4TB/Downloads:
0/0 items need repair
[ownership-repair] 3/3 config: 0/0 items need repair
[ownership-repair] complete: 0 item(s) repaired; record logs/
ownership/repair-changes.ndjson; marker logs/ownership/repair-
marker.json
```

A second invocation the same day short-circuited on the marker: `2026-08-21T17:20:37 [ownership-repair] already complete for this scope (marker: …) — nothing to do.`

Run 2 — 2026-08-25T20:52:21+02:00:

```
[ownership-repair] operator 1000:1000; scope /run/media/
milosvasic/DATA4TB/Projects/boba/config/owned_paths.yaml (6
```

```
declared locations)
[ownership-repair] 1/6 /run/media/milosvasic/DATA4TB/Projects/
boba/config/boba.db: 0/0 items need repair
[ownership-repair] 2/6 /run/media/milosvasic/DATA4TB/Projects/
boba/.env: 0/0 items need repair
[ownership-repair] 3/6 /run/media/milosvasic/DATA4TB/Downloads:
0/0 items need repair
[ownership-repair] 4/6 /run/media/milosvasic/DATA4TB/Projects/
boba/config: 0/0 items need repair
[ownership-repair] 5/6 /run/media/milosvasic/DATA4TB/Projects/
boba/tmp: 0/0 items need repair
[ownership-repair] 6/6 /run/media/milosvasic/DATA4TB/Projects/
boba/download-proxy: 0/0 items need repair
[ownership-repair] complete: 0 item(s) repaired; no change record
(nothing was changed); marker logs/ownership/repair-marker.json
```

`logs/ownership/` holds only `repair-marker.json` (mode 600) — no change record ever written, consistent with `items_changed: 0`:


```
{
  "completed_at": "2026-08-25T18:52:21Z",
  "scope_fingerprint":
        "c41619d212648a692bca539c1b3836d136b5c5c54058dacc0f81680c70120c3b",
  "items_changed": 0,
  "record_file": null
}
```


---

4. Timeline

When	Event	Source
2026-08-21 15:49 / 16:06	provenance written — **51 recorded**	`6fe921b`, `66effba`
2026-08-21 16:40:41	**`PUID=0` lands**	`c344ab0`
2026-08-21 17:01:53	`boba-stack.service` starts	journal
2026-08-21 17:01:55	run 1 — `3/3 config: 0/0`	journal
2026-08-21 17:01:56	search plugins installed	journal
2026-08-21 18:04:24–33	42 `__pycache__/*.pyc` written, uid 1000	filesystem
2026-08-25 20:52:21	run 2 — 6/6 at `0/0`	journal
2026-08-26		above

When	Event	Source
	this investigation — 0 at 100999	

The 51 ceased to exist within the **55 minutes** between the measurement and run 1's walk — a window that contains the `PUID=0` commit.

---

5. Hypotheses, adjudicated

H2 — scope mismatch — REFUTED

`git show 66effba:config/owned_paths.yaml` (the revision live at run 1) declares exactly three paths, the second being `- path: "config"` — the same literal path the provenance measured. Run 1 walked it and printed `3/3 config: 0/0`.

H3 — the walk is blind (§11.4.201(6) FALSE-NULL) — REFUTED

Four independent checks:

1. **Predicate is sound.** `scripts/ownership_repair.sh:963` builds `find … \( ! -uid "${OP_UID}" -o ! -gid "${OP_GID}" \) -printf '%U\t%G\t%m\t%p\0'` — it matches anything whose uid **or** gid differs from the operator's. Lines 967–978 capture find's stderr and treat a non-zero find status as a real condition rather than converting it into a silent zero.
2. **Relative-path resolution is not a trap here.** The script's header records that an earlier revision passed relative declared paths to `find` verbatim, resolving them against the caller's cwd — and run 1's label printed the *relative* `config`, so this revision was the affected one. It does not matter: `systemctl --user cat boba-stack.service` gives `WorkingDirectory=/run/media/milosvasic/DATA4TB/Projects/boba`, so relative `config` resolved to the project's own `config/` — the measured path.
3. **Run 1's downloads leg carried no path ambiguity at all** (absolute `/run/media/…/Downloads`) and also returned `0/0`, against an R6 claim of a 100999-owned root plus one content item.

4. **No permission wall and no unreadable filesystem** — today's walks returned empty stderr on btrfs, and the control needle sees 100999.

H4 — the provenance is wrong — NOT PROVEN FALSE, BUT UNREPRODUCIBLE

R6 records no command (§1). The 51 cannot be re-derived from the record. This is a genuine defect in the provenance's evidentiary quality and is stated as such — but it is **not** a demonstration that the number was false, and this document does not claim one.

H1 — the 51 are historical — HOLDS, with the mechanism partly established

The decisive measurement distinguishes a **rewrite** (content replaced → mtime resets) from a **chown** (ownership changes → mtime preserved):

```
config/ entries whose mtime PREDATES the 2026-08-21 16:06
measurement
  count:                       140
  of those, uid 1000 today:    140
  of those, uid != 1000:         0
```

Those 140 include files only the qBittorrent container writes, with untouched mtimes from long before `PUID=0`:

```
1000 2026-04-27 config/qBittorrent/categories.json
1000 2026-04-27 config/qBittorrent/rss/feeds.json
1000 2026-04-27 config/qBittorrent/watched_folders.json
1000 2026-04-29 config/qBittorrent/rss/storage.lock
1000 2026-08-08 config/qBittorrent/GeoDB/dbip-country-lite.mmdb
1000 2026-08-13 config/qBittorrent/logs/qbittorrent.log.bak
```

Content untouched, ownership now the operator's ⇒ **ownership changed without a rewrite**, i.e. a `chown`, not regeneration under `PUID=0`.

It was not the repair, and not project tooling:

- both runs report `0 item(s) repaired`; the marker records `items_changed: 0`, `record_file: null`; no change record exists on disk;
- `grep -n chown start.sh setup.sh install.sh install-plugin.sh stop.sh` → **no matches**; `grep -rn chown scripts/*.sh` outside `ownership_repair.sh` returns only two prose comments.

Agent and exact time of the chown: `UNKNOWN:` — see §6.

---

6. Why the chown cannot be dated — an instrument limitation, stated

A `chown` moves **ctime** while preserving mtime, so ctime is the natural instrument for dating it. It is unavailable here: a later bulk metadata event overwrote every ctime in the tree.

```
ctime day histogram, config/            ctime minute histogram,
config/
    256 2026-08-25                          246 20:52
      8 2026-08-26                            5 20:53
```

246 of 264 entries carry ctime **2026-08-25 20:52:18**, many sharing the nanosecond `.1112049470`, while their mtimes still span 2026-03-09 → 2026-08-26. A search for any ctime inside the 2026-08-21 16:00–17:02 disappearance window returns **0 entries** — not because no chown happened then, but because nothing that old survives in ctime.

Cause of the 20:52:18 bulk event: `UNKNOWN:`. It coincides to the second with the ownership precondition's start (`20:52:18 Ownership precondition: scope …`). The obvious candidate — podman's recursive SELinux relabel — is **refuted by measurement**: `docker-compose.yml` mounts `./config:/config` at lines 48, 165, 247 and 307 with **no `:Z` or `:z` suffix** on any of them. No mechanism was established, and none is asserted.

---

7. Verdict

H1 holds as to outcome; H2 and H3 are refuted; H4 is confirmed only in the narrow sense that the figure is unreproducible from the record.

Established as fact:

1. There are **no uid-100999 items today** in `config/` (0 of 264) or in the declared download root (0 of 7681), measured with a control-needled instrument on an ownership-persisting filesystem with no permission wall.
2. The repair **never chowned anything** in either real run, and no project script contains a `chown` outside it.

3. The wrongly-owned items were removed by a **chown, not a rewrite** — 140 pre-measurement files retain their original mtimes and are uid 1000.
4. That chown happened **before 2026-08-21T17:01:55**, bounded by run 1's own walk.
5. The provenance's `51` carries **no command** and cannot be re-derived.

Left honestly open:

- `UNKNOWN:` — **who** performed the chown and **exactly when** inside the 16:06→17:01:55 window. R6 itself notes the operator had been "reassigning ownership by hand", which is the only remaining candidate known to this investigation, but no captured evidence names an agent, and none is asserted.
- `UNKNOWN:` — what caused the 2026-08-25 20:52:18 bulk ctime event.

What would settle the remainder

- Shell history for uid 1000 covering 2026-08-21 16:06–17:02 (`~/.bash_history` carries no timestamps unless `HISTTIMEFORMAT` was set; `~/.zsh_history` does).
- A btrfs snapshot or backup of the tree predating 2026-08-21 17:01:55 — its owner uids would confirm or refute the 51 directly, at its stated scope.
- An audit rule (`auditctl -a always,exit -F arch=b64 -S chown,fchownat`) would date future events; it cannot recover this one.
- Operator recollection (§11.4.66) — the cheapest path, and the only one likely to name the agent.